Process solutions

State transitions, conditions, and general arguments remain part of the answers.

OS03-E01 State transitions

1. P1 running→waiting on I/O. Scheduler selects P2, ready→running. Disk completion makes P1 waiting→ready.
2. P1's blocking can switch to P2. Completion can trigger a switch if policy permits preemption. Nonpreemptive P2 may continue until blocking/yielding.
3. No. P1 is only ready. Scheduler selection and dispatcher restoration must precede running.

The single-CPU model permits at most one running process.

OS03-E02 Zombie and orphan

1. A terminated child whose parent has not collected its status is a zombie. Fork or exec alone does not create one.
2. A living child whose parent exits is orphaned and normally receives a new guardian under the OS policy.
3. Handle child exit and use wait-like operations. Signals or a centralized reaper can collect multiple children without blocking the main service indefinitely.

Zombie describes terminated state; orphan describes a parent relationship.

OS03-E03 Bounded message channel and termination

1. Blocking J3 waits for a receive to free a slot. Nonblocking send returns would-block without changing the queue. Successful send means enqueued, not completed job effect.
2. No, the specified operation is an atomic whole message. Byte streams instead accumulate returned bytes and reconstruct records. Actual pipe rules additionally depend on PIPE_BUF, length, and blocking mode.
3. Stop new jobs, send FIFO STOP, complete earlier jobs, consume STOP, acknowledge, then clean up. Two consumers with one STOP may leave one blocked forever. Use one STOP per consumer or a closed/broadcast protocol. Native remove provides no automatic drain contract.

Keep one message contract throughout.

Process solutions (continued)

OS03-E04 Context and object identity

1. Save P100's s40/7/9000 in its task-related context, then load P101's s120/3/A000 and memory context. Later P100 restores its own values. ISA rules determine completion/restart boundaries at saved PC.
2. No. Read-only code frames can be shared while private globals, heap, and stack differ. Equal VAs can map different frames; equal fd numbers can name different objects. A PCB references resources without containing the entire heap.
3. No. One process's entry/return may change only mode. Its threads can switch registers/stack while sharing user AS. Mode, task, and AS changes are distinct.

Check identity at each intermediate state.

OS03-E05 Lost wakeup

1. Waiter checks false; handler sets done=true but finds no queued waiter; waiter then enqueues/sleeps. With no later completion, it waits forever.
2. Under L, the waiter checks done, enqueues and sets WAITING if false, then unlocks. It sleeps only while still WAITING. Handler under L sets done and changes a waiter to READY. Completion before check means no sleep. During L it waits until registration. After actual sleep it finds/wakes the waiter. Between unlock and schedule, it sets READY, which schedule must preserve rather than sleeping unconditionally.
3. Wakeup establishes ready eligibility. Selection/dispatch still follow, and a nonpreemptive current task may continue.

The common lock, predicate, and state transition jointly prevent loss.

OS03-E06 Fork tree and lifecycle

1. First P0→P1, then P0→P2 and P1→P3, total4. If every current process reaches each successful fork, N(k+1)=2N(k), N0=1, hence2^k by induction. Parent-only continuation, child exec/exit, or failure changes the recurrence.
2. O1 moves0→100→150. Child starts reading at100. Independent fd-table entries can share one description. Separately opening the same inode creates an independent O2 offset, possibly0. This is unrelated to private-page COW.
3. Exec replaces one process image without increasing process count. Exit7 leaves collectible zombie status until wait. A living child after parent exit is orphaned. Wait must first recognize normal exit and decode7; its raw status word is not simply exit code7.

Process solutions (continued)

Separate execution, resource, and status lifetimes.

OS03-E07 SPSC ring and publication

1. (0,0)[]→(0,1)[A]→(0,2)[A, B]. Failed C leaves state unchanged. Get A→(1,2)[B]; put C→(1,0)[B, C]; get B→(2,0)[C]; get C→(0,0)[]. Capacity is2.
2. Write slot before release tail; consumer's corresponding acquire sees completed data. Read slot before release head; producer's acquire allows reuse only after consumption. Empty/full and unique writers preserve0≤size≤2 and FIFO.
3. No. Plain conflicting C/C++ accesses can race and lack ordering. Two producers compete for one tail/slot and violate the unique-writer premise. Cross-process atomics also need explicit platform support, not merely a C++ type.

Finite sequential traces support index checks, not a replacement for the ordering argument.

OS03-E08 Framing and shared objects

1. First chunk is half a header, no output. Second establishes length 3 but only AB, no output. Third adds C, emits ABC, then decodes length 2 and emits DE. Read boundaries are not record boundaries.
2. After four bytes, header3 has only AB, so EOF is truncation. Reject length 9 above max8 before blindly allocating. EOF exactly at a complete-record boundary may terminate normally.
3. VAs are0x100010 and0x700010, possibly the same backing. Offsets work across different bases; raw pointers do not. Close does not unmap; shm_unlink removes a name without instantly invalidating existing mappings. Initialization/publication still requires synchronization.
4. No. mq_unlink removes the name while open users continue until final close. IPC_RMID removes the queue and wakes blocked operations with EIDRM. STOP/drain is a separate application protocol.

Keep names, handles, mappings, and protocol completion distinct.

Process solutions (continued)

OS03-E09 RPC retry and encoding

1. 00 00 00 4D | 00 01 | 00 00 00 19, total10 B. Specify widths/byte order without native padding or pointers.
2. A never receives the request, balance 100. B commits75 but loses reply. Both produce client timeout. A new-ID retry makes B50; no retry leaves A undone. Timeout cannot establish nonexecution.
3. Index by(client identity, id). Update balance and result 75 in one recoverable atomic transaction. Repeated77 returns 75 without another debit. Concurrent duplicates need transaction/unique-constraint serialization rather than two absent checks and two effects.
4. RAM records vanish on restart. Debit before ID recording leaves a crash window. Expired IDs with permitted old retries can execute again. Require joint durable commit, ID/epoch scope, and retention agreements. At-most-once allows zero effects; it is not unconditional exactly-once.

The effect and dedup result must share one commit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Process solutions (continued)

Verification code appendix: process_ipc_models.py

```
"""Sequential teaching models only; not C++ atomics or persistent database code."""

class Ring:
    """One reserved slot; each method is one abstract atomic operation."""
    def __init__(self, slots=3):
        if slots<2:raise ValueError("at least two physical slots")
        self.slots=[None]*slots
        self.head=self.tail=0

    def put(self,item):
        next_tail=(self.tail+1)%len(self.slots)
        if next_tail==self.head:return False
        self.slots[self.tail]=item
        self.tail=next_tail
        return True

    def get(self):
        if self.head==self.tail:return False,None
        item=self.slots[self.head]
        self.head=(self.head+1)%len(self.slots)
        return True,item

    def pending(self):
        result=[];cursor=self.head
        while cursor!=self.tail:
            result.append(self.slots[cursor])
            cursor=(cursor+1)%len(self.slots)
        return result


class RecordParser:
    def __init__(self,maximum=8):
        self.buffer=bytearray();self.maximum=maximum

    def feed(self,fragment):
        self.buffer.extend(fragment)
        records=[]
        while len(self.buffer)>=2:
            length=int.from_bytes(self.buffer[:2],"big")
            if length>self.maximum:raise ValueError("record length exceeds limit")
            if len(self.buffer)<2+length:break
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Process solutions (continued)

```
            records.append(bytes(self.buffer[2:2+length]))
            del self.buffer[:2+length]
        return records

    def finish(self):
        if self.buffer:raise ValueError("truncated record at EOF")


def debit_transaction(state,client,request_id,amount):
    """Return one indivisible next-state proposal: balance and dedup record."""
    balance,records=state
    key=(client,request_id)
    if key in records:
        original,result=records[key]
        if original!=amount:raise ValueError("request ID reused for another operation")
        return state,result
    if amount<=0 or amount>balance:raise ValueError("invalid debit")
    after=balance-amount
    copied=dict(records);copied[key]=(amount,after)
    return (after,copied),after


if __name__=="__main__":
    ring=Ring()
    for item in "AB":assert ring.put(item)
    assert not ring.put("C")
    print("full:",ring.head,ring.tail,ring.pending())
    assert ring.get()==(True,"A") and ring.put("C")
    print("wrapped:",ring.head,ring.tail,ring.pending())
    parser=RecordParser();messages=[]
    for fragment in [bytes.fromhex("00"),bytes.fromhex("03 41 42"),bytes.fromhex("43 00 02 44
45")]:
        messages.extend(parser.feed(fragment))
    parser.finish();print("records:",messages)
    state=(100,{})
    state,result=debit_transaction(state,"A",77,25)
    state,repeated=debit_transaction(state,"A",77,25)
    assert result==repeated==75 and state[0]==75
    print("same-ID retry:",state,result,repeated)
```

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study